

TERM PROJECT REPORT

Student Name: Merve Adalı

Student Number: 211805049

Lesson: CSE 431 - Natural Language Processing with Machine Learning

Instructor: Assoc. Prof. Dr. Fatih Soygazi

Submission Date: 28.12.2025

1. Introduction

With the rapid growth of smart home technologies, natural language interfaces have become an essential component for enabling intuitive human–computer interaction. Voice-controlled smart home systems allow users to control devices such as lighting, television, climate control, and multimedia systems using natural language commands.

The goal of this project is to develop a Turkish smart home command recognition system that can understand user intent even when different linguistic expressions or paraphrases are used. To achieve this, Word2Vec-based word embeddings are employed to capture semantic similarity between command phrases. Sentence-level embeddings are constructed using both mean pooling and TF-IDF weighted averaging, and commands are matched using cosine similarity.

2. Dataset Creation

2.1 Core Command Definition

First, a set of core smart home commands was defined, covering common household actions such as:

- Lighting control (on, off, brightness adjustment)
- Television and multimedia control
- Climate control (heating, cooling)
- Curtains, fans, alarms, and scene commands

These core commands form the basis of the smart home system.

2.2 AI-based Command Variation Generation

For each core command, semantically similar and alternative expressions were generated using AI-based language models such as ChatGPT. These AI tools were used to create natural Turkish paraphrases that reflect how users may express the same intent in different ways.

The purpose of this step is to increase linguistic diversity and improve the robustness of the model against variations in user speech.

Example Variations

Core Command	AI-generated Variations
ışığı yak	lambayı aç, ışığı aç, odayı aydınlat
televizyonu kapat	TV'yi kapat, televizyonu kapatır mısın
klimayı aç	klimayı çalıştır, klimayı açar mısın
müziği durdur	müziği kapat, müziği sustur
alarmı kur	alarmı ayarla, alarmı başlat

All generated commands were stored in an Excel file (TR_Commands.xlsx) and later converted into a training corpus.

3. Training Corpus Preparation

The command dataset was processed and converted into a text-based corpus format. Each command was paired with an automatically assigned intent label based on keyword matching. The final corpus was saved as train_commands.txt.

- Total number of command sentences: 71
- All commands were lowercased and tokenized
- Each line consists of an intent label and the corresponding command text

This corpus was used for fine-tuning Word2Vec models.

4. Word2Vec Model Fine-tuning

4.1 Pre-trained Turkish Word2Vec Model

A pre-trained Turkish Word2Vec model provided by Akoksal et al. was used as the base embedding model. This model contains rich semantic representations trained on large Turkish corpora.

4.2 Unigram Word2Vec Fine-tuning

The pre-trained model was fine-tuned using the smart home command corpus. Only words that were common between the pre-trained vocabulary and the project dataset were initialized with pre-trained vectors.

Key parameters:

- Vector size: inherited from pre-trained model
- Window size: 5
- Epochs: 25
- Minimum word count: 1

The resulting model was saved as finetuned.model.

4.3 Bigram (Multiword) Word2Vec Fine-tuning

In the second phase, multiword expressions (bigrams) were incorporated using Gensim's Phrases and Phraser modules. Frequently co-occurring word pairs such as “*işığı aç*” were detected and treated as single tokens.

The bigram-enhanced corpus was then used to fine-tune another Word2Vec model, which was saved as `finetuned_bigram.model`.

This step enables the system to better capture phrase-level semantics.

5. Sentence Embedding Generation

Each command sentence was represented as a single fixed-length vector using two different approaches:

5.1 Mean of Word Vectors

The sentence embedding was computed as the arithmetic mean of the Word2Vec embeddings of all words in the sentence.

5.2 TF-IDF Weighted Mean

In this approach, word embeddings were weighted by their TF-IDF scores, giving more importance to informative words and reducing the influence of frequently occurring terms.

Both approaches were applied separately for:

- Unigram Word2Vec model
- Bigram Word2Vec model

All sentence embeddings were saved as NumPy arrays for later evaluation.

6. Command Matching and Decision Making

To recognize user commands, the following procedure was used:

1. The input sentence was converted into a sentence embedding.
2. Cosine similarity was computed between the input vector and all stored command vectors.
3. A confidence threshold of 0.75 was applied.
4. If the maximum similarity exceeded the threshold, the command was accepted; otherwise, it was rejected as not recognized.

This approach allows flexible matching of semantically similar commands.

7. Evaluation Setup

7.1 Test Dataset

A balanced test dataset was created, consisting of:

- 10 valid smart home commands
- 10 invalid or out-of-domain commands

Each test command was manually labeled as correct (1) or incorrect (0).

7.2 Evaluation Metrics

The performance of each approach was evaluated using:

- **Precision**
- **Recall**
- **F1-Score**

These metrics provide a comprehensive assessment of recognition accuracy.

8. Experimental Results

Unigram Model Results

Method	Precision	Recall	F1-Score
Mean	0.82	0.90	0.86
TF-IDF	0.82	0.90	0.86

Bigram Model Results

Method	Precision	Recall	F1-Score
Mean	0.82	0.90	0.86
TF-IDF	0.82	0.90	0.86

9. Discussion

The results show that both unigram and bigram Word2Vec models perform well in recognizing Turkish smart home commands. The use of TF-IDF weighting did not significantly outperform simple mean pooling in this dataset, likely due to the short length of command sentences.

The bigram model provides a more linguistically meaningful representation, although its quantitative improvement is limited by the dataset size.

10. Conclusion

In this project, a Turkish smart home command recognition system was successfully implemented using Word2Vec embeddings and cosine similarity. The system is capable of recognizing semantically similar command variations and rejecting unrelated inputs.

Future work may include:

- Larger and more diverse datasets
- Real-time speech-to-text integration
- Deep learning-based sentence embeddings

References

- [1] Akoksal, A. – Turkish Word2Vec
<https://github.com/akoksal/Turkish-Word2Vec>
- [2] Gensim Word2Vec Documentation
<https://radimrehurek.com/gensim/models/word2vec.html>
- [3] Scikit-learn TF-IDF Vectorizer
https://scikit-learn.org/1.5/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- [4] Gensim Phrase Detection
<https://radimrehurek.com/gensim/models/word2vec.html#embeddings-with-multiword-ngrams>
- [5] OpenAI. ChatGPT (GPT-based Large Language Model)
<https://openai.com/chatgpt>